

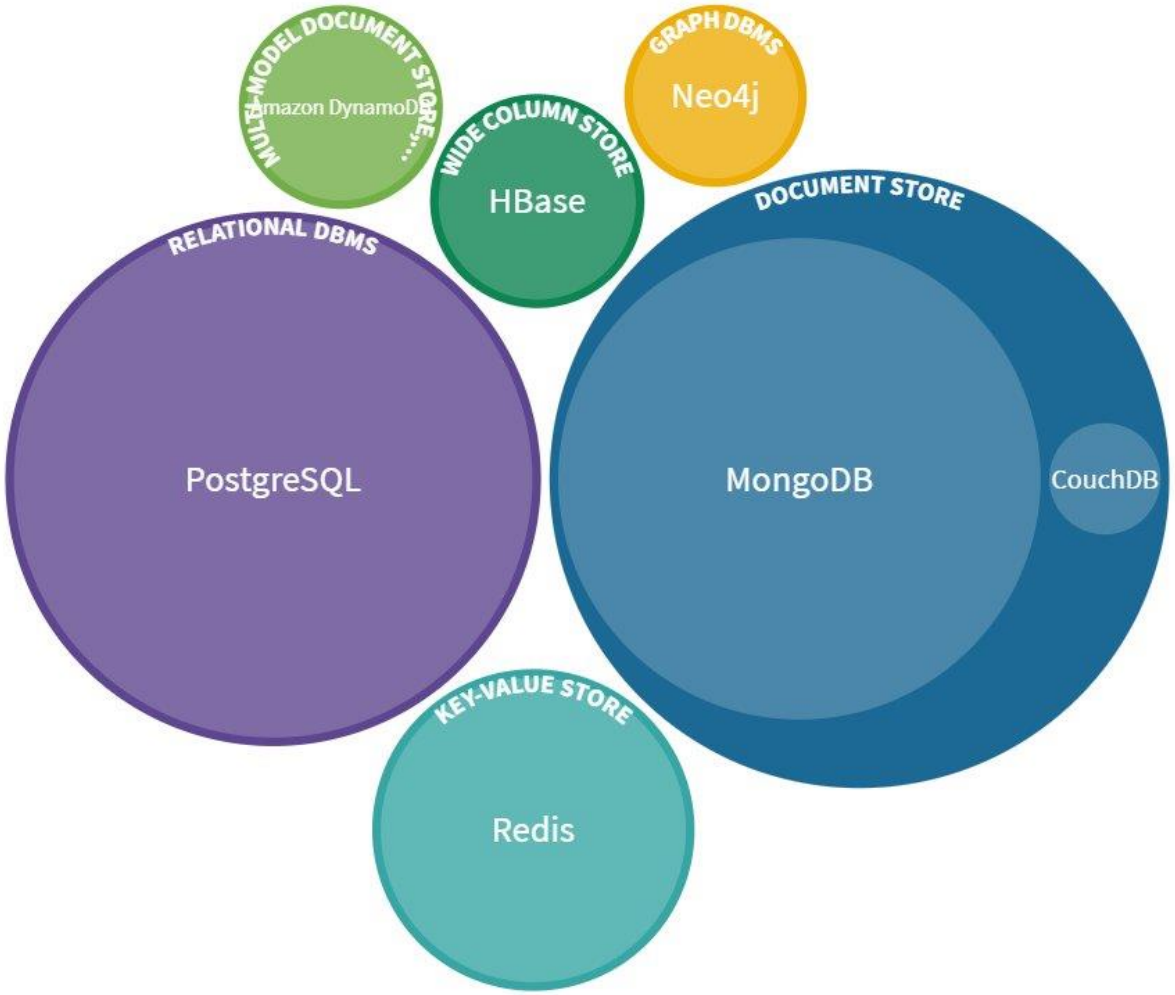
redis (**RE**mote **DI**ctionary **S**ervice)

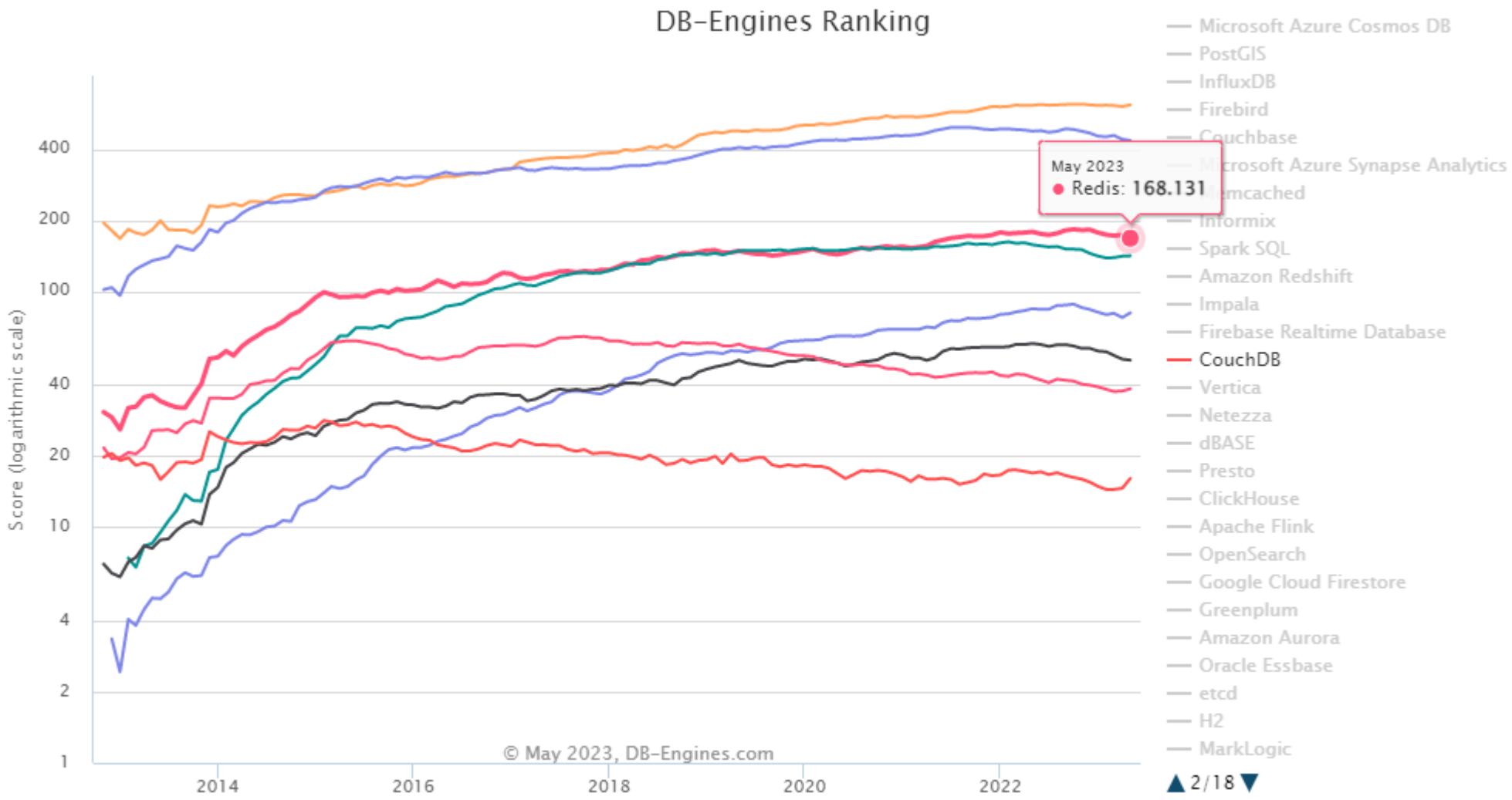
Redis is an **in-memory database** that persists on disk. The data model is key-value, but many different kinds of values are supported: Strings, Lists, Sets, Sorted Sets, Hashes, Streams, HyperLogLogs, Bitmaps.

Repository: https://github.com/redis/redis

Initial release May 10, **2009**
Stable release: 7.0.11, April 17, **2023**

DB-Ranking (**Key-value store**)
#6 Overall
#1 Key-value stores


MULTI-MODEL DOCUMENT STORE
Amazon DynamoDB
WIDE COLUMN STORE
HBase
GRAPH DBMS
Neo4j
DOCUMENT STORE
RELATIONAL DBMS
PostgreSQL
MongoDB
CouchDB
KEY-VALUE STORE
Redis



Redis (**Re**mote **Di**ctionary **S**erver) is an open-source in-memory data structure project implementing a distributed, in-memory key-value database *with optional durability*. Redis supports different kinds of abstract data structures, such as strings, lists, maps, sets, sorted sets,

hyperloglogs, bitmaps, streams and spatial indexes. The project is mainly developed by Salvatore Sanfilippo and is currently sponsored by Redis Labs.

The end of the Redis adventure - <antirez>

Redis has built-in replication, transactions and different levels of on-disk persistence, and provides high availability via Redis Sentinel and automatic partitioning with Redis Cluster. https://redis.io/

First released in 2009, is a **simple-to-use** key-value store with a **sophisticated set of commands**

Redis is hard to beat, Reads are fast and writes are even faster, handling upwards of **100,000 SET operations per second** by some benchmarks.

Instalando Redis (en la practica usen el docker)

```
$ wget http://download.redis.io/redis-stable.tar.gz
$ tar xvzf redis-stable.tar.gz
$ cd redis-stable
$ make
```

Para hacer el `make test`, tienen que tener una versión actualizada del **tcl**

```
$ sudo apt-get install tcl
$ cd src && make test
```

Si todo va bien, obtenemos un: **All tests passed without errors!**

Levantando el servidor/servicio.

```
$ cd ~
$ redis-stable/src/redis-server
```

Por default, corre en el puerto **6379**.

instalando Redis en Windows con WSL2

```
curl -fsSL https://packages.redis.io/gpg | sudo gpg --dearmor -o /usr/share/keyrings/redis-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/redis-archive-keyring.gpg] https://packages.redis.io/deb $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/redis.list

sudo apt-get update
sudo apt-get install redis

sudo service redis-server start
```

Levantando el cliente (CLI) en otra terminal.
En el mismo directorio donde está el redis-server, encontramos el redis-cli.

```
$ redis-stable/src/redis-cli

127.0.0.1:6379> PING
PONG

127.0.0.1:6379> HELP
redis-cli 7.0.11
To get help about Redis commands type:
      "help @<group>" to get a list of commands in <group>
      "help <command>" for help on <command>
      "help <tab>" to get a list of possible help topics
```

```
      "quit" to exit
```

https://redis.io/topics/data-types

El libro, propone implementar un servicio, para acortar URLs, al estilo **bit.ly**

SET y GET
Time complexity: O(1)

```
127.0.0.1:6379> SET 7wks http://www.sevenweeks.org/
OK

127.0.0.1:6379> GET 7wks
"http://www.sevenweeks.org/"
```

MSET y MGET
Time complexity: O(N) where N is the number of keys to retrieve.

```
127.0.0.1:6379> MSET gog http://www.google.com yah http://www.yahoo.com
OK

127.0.0.1:6379> MGET gog yah 7wks
1) "http://www.google.com"
2) "http://www.yahoo.com"
3) "http://www.sevenweeks.org/"
```

Sumando uno a una clave
INCR *key*

```
127.0.0.1:6379> SET count 10
```

```
OK
127.0.0.1:6379> INCR count
(integer) 11
127.0.0.1:6379> GET count
"11"
```

Transacciones
MULTI, **EXEC** and **DISCARD** are the foundation of transactions in Redis

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379> SET prag http://pragprog.com
QUEUED
127.0.0.1:6379> INCR count
QUEUED
127.0.0.1:6379> EXEC
1) OK
2) OK
```

El **DISCARD** es similar al ROLLBACK de SQL, pero lo que hace es no ejecutar la transacción, no vuelve los datos al estado anterior.
(*transaction rollback vs. operation cancellation*)

Complex data types
lists, hashes, sets, sorted sets, etc.

Estas colecciones, pueden almacenar hasta $2^{\wedge 32}$ por clave. (4,294,967,296)
Cuanta memoria necesitamos dependerá de las colecciones que vamos a manejar. What's the Redis memory footprint?

S**ETs commands begin with **S
H**ashes with **H

S**orted Sets with **Z
***L**ist commands generally start with either an **L** (for left) or an **R** (for right), depending on the direction of the operation (such as LPUSH).*

Hashes

```
127.0.0.1:6379> MSET user:luc:name "Luc" user:luc:password "s3cret"
OK
127.0.0.1:6379> MGET user:luc:name user:luc:password
1) "Luc"
2) "s3cret"
```

En lugar de claves separadas, podemos crear un hash que contenga sus propios pares clave-valor.

```
127.0.0.1:6379> HMSET user:luc name "Luc" password "s3cret"
OK
```

Para recuperar las claves almacenadas, usamos **HKEYS**. (en el libro usan HVALS y está mal. https://pragprog.com/titles/pwrdata/errata)

```
127.0.0.1:6379> HKEYS user:luc
1) "name"
2) "password"
```

Podemos pedir, todos los elementos de la hash key

```
127.0.0.1:6379> HVALS user:luc
1) "Luc"
2) "s3cret"
```

o sólo alguno en particular

```
127.0.0.1:6379> HGET user:luc password
"s3cret"

127.0.0.1:6379> HGETALL user:luc
…
```

otros comandos, HDEL, HINCRBY, HLEN, HGETALL, HSETNX, etc.

Si quiero saber las claves que tengo creadas hasta el momento.

```
127.0.0.1:6379> KEYS *
1) "7wks"
2) "yah"
3) "count"
4) "gog"
5) "user:luc"
...
```

Listas

Las listas contienen múltiples valores ordenados que pueden actuar como **colas** (*first in, first out*) y como **pilas** (*last in, first out*). También tienen acciones más sofisticadas para insertar en algún lugar en medio de una lista, restringir el tamaño de la lista y mover valores entre listas.

RPUSH

```
127.0.0.1:6379> RPUSH eric:wishlist 7wks gog prag
(integer) 3
```

LRANGE

```
127.0.0.1:6379> LRANGE eric:wishlist 0 -1
1) "7wks"
```

```
2) "gog"
3) "prag"
```

LREM

```
127.0.0.1:6379> LREM eric:wishlist 0 gog
(integer) 1

127.0.0.1:6379> LRANGE eric:wishlist 0 -1
1) "7wks"
2) "prag"
```

LPOP

```
127.0.0.1:6379> LPOP eric:wishlist
"7wks"
127.0.0.1:6379> LPOP eric:wishlist
"prag"
127.0.0.1:6379> LPOP eric:wishlist
(nil)
```

To act as a ***stack***, after you **RPUSH** the values, you would **RPOP** from the end of the list
You can use **LPUSH** and **RPOP** to similar effect (a ***queue***) or **LPUSH** and **LPOP** to be a stack.

Redis provides a single command for popping values from the tail of one list and pushing to the head of another. It's called **RPOPLPUSH** (right pop, left push).

```
127.0.0.1:6379> RPOPLPUSH eric:wishlist eric:visited
"prag"
```

Blocking Lists

BRPOP is a blocking list pop primitive
BRPOP key [key ...] timeout
It is the blocking version of **RPOP** because it blocks the connection when there are no elements to pop from any of the given lists.

```
127.0.0.1:6379> BRPOP comments 60
```

Me quedo esperando 60 segundos para que aparezca un comentario.
Si en otra sesión, alguien inserta un comentario.

11 segundos después, desde otra terminal.

```
127.0.0.1:6379> LPUSH comments "hola mundo"
(integer) 1
```

en mi terminal, veo esto:

```
127.0.0.1:6379> BRPOP comments 60
1) "comments"
2) "hola mundo"
(11.10s)
```

Sets

SADD key member [member ...]

```
127.0.0.1:6379> SADD news nytimes.com pragprog.com
(integer) 2
```

SMEMBERS key

```
127.0.0.1:6379> SMEMBERS news
1) "pragprog.com"
2) "nytimes.com"
```

```
127.0.0.1:6379> SADD tech pragprog.com apple.com
(integer) 2
```

Intersección (de conjuntos)
SINTER key [key ...]

```
127.0.0.1:6379> SINTER news tech
1) "pragprog.com"
```

Diferencia
SDIFF key [key ...]

```
127.0.0.1:6379> SDIFF news tech
1) "nytimes.com"
```

Union
SUNION key [key ...]

```
127.0.0.1:6379> SUNION news tech
1) "apple.com"
2) "pragprog.com"
3) "nytimes.com"
```

Si queremos guardar la unión de los conjuntos en una nueva clave, usamos **SUNIONSTORE**
SUNIONSTORE destination key [key ...]

```
127.0.0.1:6379> SUNIONSTORE websites news tech
127.0.0.1:6379> SMEMBERS websites
1) "pragprog.com"
2) "nytimes.com"
3) "apple.com"
```

Así como está el **SUNIONSTORE**, existe también el **SINTERSTORE** y el **SDIFFSTORE**.

SMOVE source destination member
El SMOVE sirve para mover elementos de un conjunto a otro.

Así como el **LLEN** te da la cantidad de elementos de una lista, el SCARD (S de Sets, CARD de cardinalidad), te da la cantidad o cardinalidad de elementos de un conjunto.

Debido a que los conjuntos no están ordenados, no hay comandos de izquierda, derecha u otros comandos posicionales. Para tomar un elemento aleatorio de un conjunto usamos **SPOP**, y eliminar **SREM**.

SPOP	SREM
redis> SADD myset "one" (integer) 1 redis> SADD myset "two" (integer) 1 redis> SADD myset "three" (integer) 1 redis> **SPOP** myset "one" redis> SMEMBERS myset 1) "three" 2) "two"	redis> SADD myset "one" (integer) 1 redis> SADD myset "two" (integer) 1 redis> SADD myset "three" (integer) 1 redis> **SREM** myset "one" (integer) 1 redis> **SREM** myset "four" (integer) 0 redis> SMEMBERS myset 1) "three" 2) "two"

Sorted Sets

Los conjuntos ordenados de Redis son, al igual que los conjuntos de Redis, colecciones de cadenas que no se repiten. La diferencia es que cada miembro de un conjunto ordenado está asociado con un ***score***, que se utiliza para tomar el conjunto de manera ordenada, desde la puntuación más pequeña a la más alta. Mientras que los miembros son únicos, las puntuaciones pueden repetirse.

Con los *Sorted Sets* se puede implementar una tabla de posiciones en un juego masivo en línea, donde cada vez que se envíe un nuevo puntaje, lo actualizamos con ZADD y tomar los mejores jugadores con ZRANGE.

Volviendo a nuestro ejemplo, de las URLs, vamos a agregar tres elementos con sus valores (de score).

ZADD key [NX|XX] [CH] [INCR] score member [score member ...]

```
127.0.0.1:6379> ZADD visits 500 7wks 9 gog 9999 prag
(integer) 3
```

visits es la clave y después agregamos 3 elementos 'score member' al mismo tiempo.

Para incrementar un score de un elemento de este conjunto (usamos el ZINCRBY)

```
127.0.0.1:6379> ZINCRBY visits 1 prag
"10000"
```

Si queremos acceder a los primeros 2 elementos de este conjunto

```
127.0.0.1:6379> ZRANGE visits 0 1
1) "gog"
2) "7wks"
```

ZRANGE (de menor a mayor) y ZREVRANGE (REV = reverso)

```
127.0.0.1:6379> ZREVRANGE visits 0 -1 WITHSCORES
```

```
1) "prag"
2) "10000"
3) "7wks"
4) "500"
5) "gog"
6) "9"
```

Si queremos solo los que el score esté en este intervalo [9 .. 10000].

```
127.0.0.1:6379> ZRANGEBYSCORE visits 9 10000
1) "gog"
2) "7wks"  3) "prag"
```

Si no queremos incluir al 9, es decir (9..10000] la consulta sería la siguiente:

```
127.0.0.1:6379> ZRANGEBYSCORE visits (9 9999
1) "7wks"
```

También podemos usar inf y -inf

```
127.0.0.1:6379> ZRANGEBYSCORE visits -inf inf
1) "gog"
2) "7wks"
3) "prag"

127.0.0.1:6379> ZREVRANGEBYSCORE visits inf -inf
1) "prag"
2) "7wks"
3) "gog"
```

Unions
ZUNIONSTORE destination numkeys key [key ...] [WEIGHTS weight [weight ...]] [AGGREGATE SUM|MIN|MAX]

Podemos unir conjuntos y asignarle diferente peso a los elementos de cada conjunto.

EXPIRE key seconds

Podemos crear un elemento y darle un tiempo de vida (time to live o TTL)

```
127.0.0.1:6379> SET ice "I'm melting..."
OK
127.0.0.1:6379> EXPIRE ice 10
(integer) 1
127.0.0.1:6379> EXISTS ice
(integer) 1
127.0.0.1:6379> EXISTS ice
(integer) 0
```

Antes de que 'ice' desaparezca, podemos preguntar cuánto le falta.

```
127.0.0.1:6379> TTL ice
(integer) 4
```

En cualquier momento, antes de que el dato expire, podemos anular el TTL usando PERSIST.

```
127.0.0.1:6379> PERSIST ice
```

Database Namespaces

0 → es la default.

Para cambiar de base, usar SELECT index.

En términos prácticos, las bases de datos de Redis se deben utilizar principalmente si es necesario separar diferentes claves que pertenecen a la misma aplicación y no para usar una sola instancia de Redis para múltiples aplicaciones no relacionadas.

Cuando se usa Redis Cluster, el comando SELECT no se puede usar, ya que Redis Cluster solo admite la base de datos 0 (cero).

Ejemplo

```
127.0.0.1:6379> SET greeting hello
OK
127.0.0.1:6379> GET greeting
"hello"



127.0.0.1:6379> SELECT 1
OK
127.0.0.1:6379[1]> GET greeting
(nil)

127.0.0.1:6379[1]> SET greeting "guten Tag"
OK
127.0.0.1:6379[1]> SELECT 0
OK
127.0.0.1:6379> GET greeting
"hello"
```

Podemos mover keys entre bases de datos.

```
127.0.0.1:6379> MOVE greeting 2
(integer) 2
127.0.0.1:6379> SELECT 2
OK
127.0.0.1:6379[2]> GET greeting
"hello"
```

Cuidado con estos dos comandos:
FLUSHDB
Delete all the keys of the currently selected DB

FLUSHALL
Delete all the keys of all the existing databases, not just the currently selected one

Tarea

Install your favorite programming language driver and connect to the Redis server. Insert and increment a value within a transaction.

https://pypi.org/project/redis/

```
$ sudo pip install redis
```

Getting Started

```
>>> import redis
>>> r = redis.StrictRedis(host='localhost', port=6379, db=0)
>>> r.set('foo', 'bar')
True
```

```
>>> r.get('foo')
'bar'
```

```
MULTI/EXEC: These are implemented as part of the Pipeline class. The pipeline is wrapped with the MULTI and EXEC
statements by default when it is executed, which can be disabled by specifying transaction=False
```

```
>>> r = redis.Redis()
>>> r.set('bing', 'baz')
>>> # Use the pipeline() method to create a pipeline instance
>>> pipe = r.pipeline()
>>> # The following SET commands are buffered
>>> pipe.set('foo', 'bar')
>>> pipe.get('bing')
>>> # the EXECUTE call sends all buffered commands to the server, returning
>>> # a list of responses, one for each command.
>>> pipe.execute()
[True, 'baz']
```

Otra manera de llamar al pipeline.

```
>>> pipe.set('foo', 'bar').sadd('faz', 'baz').incr('auto_number').execute()
```

FIN dia 1

Redis implementa REST?
HTTP/REST = no

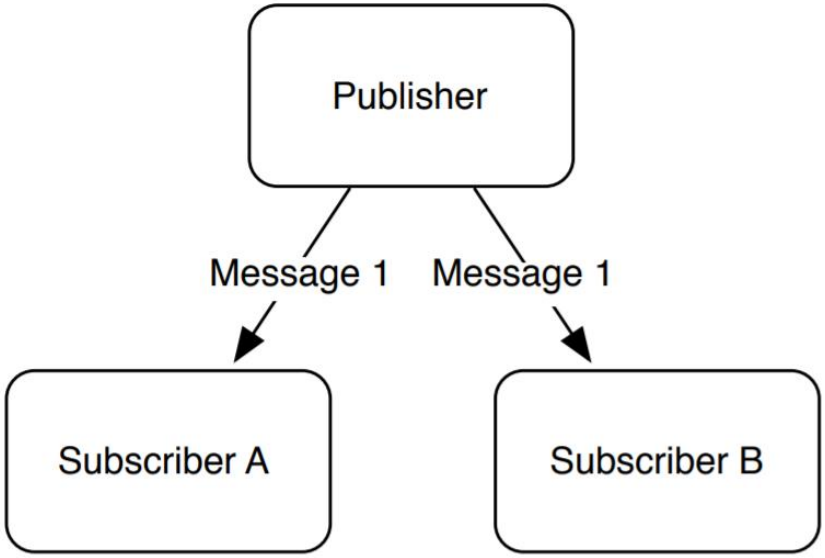
En cambio, tenemos lo siguiente:
Interface Protocol = Simple text over TCP

Podemos interactuar con redis sin usar el redis-cli, directamente con comandos TCP.
Por ej, con **telnet** o con el **nc** (netcat)

telnet	Pipelining usando nc (netcat)
redis/telnet.sh **$ telnet localhost 6379** Trying 127.0.0.1... Connected to localhost. Escape character is '^]'. **SET test hello** +OK **GET test** $5 hello **SADD stest 1 999** :2 **SMEMBERS stest** *2 $1 1 $3 999 para salir del telnet Ctrl +], quit	**$ (echo -en "ECHO hello\r\n"; sleep 1) | nc localhost 6379** $5 hello **$ (echo -en "PING\r\nPING\r\nPING\r\n"; sleep 1) | nc localhost 6379** +PONG +PONG +PONG

---

publish-subscribe
Pub/Sub



Abrir dos clientes **redis** y suscribirse al canal 'comments'.

```
SUBSCRIBE channel [channel ...]
```

```
127.0.0.1:6379> SUBSCRIBE comments
Reading messages... (press Ctrl-C to quit)
1) "subscribe"
2) "comments"
3) (integer) 1
```

Desde otro cliente, publicar un comentario.

```
127.0.0.1:6379> PUBLISH comments "Check out this shortcoded site! 7wks"
(integer) 2
```

El `'(integer) 2'` es porque este mensaje lo van a ver los suscriptores.
En los dos clientes, vamos a ver el mensaje recién publicado.

```
1) "message"
2) "comments"
3) "Check out this shortcoded site! 7wks"
```

Server Info

El comando INFO devuelve información y estadísticas sobre el servidor.

```
127.0.0.1:6379> INFO
# Server
redis_version:5.0.0
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:7907ef6bdcb301be
redis_mode:standalone
os:Linux 3.16.0-77-generic x86_64
arch_bits:64
...
```

Redis Configuration

Si lanzamos Redis sin ningún archivo de configuración, nos aparece este Warning.

```
# Warning: no config file specified, using the default config. In order to specify a config file use redis-stable/src/redis-server /path/to/redis.conf

drwxr-xr-x  6 ariel ariel  4096 Oct 17 12:31 ./
drwxr-xr-x 19 ariel ariel  4096 Nov  4 14:56 ../
-rw-r--r--  1 ariel ariel 75104 Oct 17 12:31 00-RELEASENOTES
-rw-r--r--  1 ariel ariel    53 Oct 17 12:31 BUGS
-rw-r--r--  1 ariel ariel  1894 Oct 17 12:31 CONTRIBUTING
-rw-r--r--  1 ariel ariel  1487 Oct 17 12:31 COPYING
drwxr-xr-x  6 ariel ariel  4096 Nov  3 09:53 deps/
-rw-r--r--  1 ariel ariel   376 Oct 17 12:31 .gitignore
-rw-r--r--  1 ariel ariel    11 Oct 17 12:31 INSTALL
-rw-r--r--  1 ariel ariel   151 Oct 17 12:31 Makefile
-rw-r--r--  1 ariel ariel  4223 Oct 17 12:31 MANIFESTO
-rw-r--r--  1 ariel ariel 20555 Oct 17 12:31 README.md
-rw-r--r--  1 ariel ariel 62155 Oct 17 12:31 redis.conf
-rwxr-xr-x  1 ariel ariel   275 Oct 17 12:31 runtest*
-rwxr-xr-x  1 ariel ariel   280 Oct 17 12:31 runtest-cluster*
-rwxr-xr-x  1 ariel ariel   281 Oct 17 12:31 runtest-sentinel*
-rw-r--r--  1 ariel ariel  9710 Oct 17 12:31 sentinel.conf
drwxr-xr-x  3 ariel ariel  4096 Nov  3 14:18 src/
drwxr-xr-x 10 ariel ariel  4096 Oct 17 12:31 tests/
drwxr-xr-x  8 ariel ariel  4096 Oct 17 12:31 utils/
```

Vamos a ver algunos parámetros, por ej.

daemonize no	Cuando estamos en modo desarrollo, 'no' es una buena opción, cuando pasamos a producción, es mejor que

	corra en modo 'daemon', en ese caso el *logfile* no puede ser el *stdout*, tiene que ser un archivo.
port 6379	6379 es el port default, pero si queremos lanzar otro redis como slave, lo podríamos poner en el 6380. En realidad si queremos tener alta disponibilidad y baja latencia, deberíamos ponerlo en otro servidor y no en la misma máquina y otro puerto.
loglevel verbose	
logfile stdout	o el *path* a un archivo.
database 16	Si sabemos que solo vamos a trabajar con la base de datos 0 y no vamos a tener ninguna más, es mejor que este parámetro lo cambiemos a '1'.

Durability (o Persistencia)

Redis tiene algunas opciones de persistencia. En primer lugar tenemos la **no persistencia**, lo que simplemente mantendrá todos los valores en la memoria principal.

Por otro lado, por default Redis graba (cada tanto) la información que tiene en memoria la persiste al disco cada tanto. Los valores por defaults son los siguientes:

Seccion SAVE del redis.conf → http://download.redis.io/redis-stable/redis.conf

```
# save <seconds> <changes>
```

```
save 900 1
save 300 10
save 60 10000

#   In the example below the behavior will be to save:
#   after 900 sec (15 min) if at least 1 key changed
#   after 300 sec (5 min) if at least 10 keys changed
#   after 60 sec if at least 10.000 keys changed
```

Si queremos obligarlo a grabar los datos, tenemos 2 comandos (SAVE y BGSAVE)

SAVE
Nunca deberías llamar al SAVE en un entorno de producción, porque bloquearía a los otros clientes, es mejor usar el BGSAVE (**b**ack**g**round **save**)

Seguridad

https://redis.io/topics/security

Although Redis is not natively built to be a fully secure server, you may run across the requirepass setting and AUTH command in the Redis documentation.

*These can be safely ignored because they are merely a scheme for setting a **plaintext password**. Because a client can try nearly **100,000 passwords a second**, it's almost a moot point, beyond the fact that plaintext passwords are inherently unsafe anyway. If you want Redis security, you're better off with a **good firewall** and **SSH security**.*

Deshabilitando ciertos comandos, los comandos peligrosos [FLUSHDB,FLUSHALL] y el **rename-command**.

```
127.0.0.1:6379> rename-command FLUSHALL c283d93ac9528f986023793b411e4ba2
127.0.0.1:6379> FLUSHALL
```

```
(error) ERR unknown command 'FLUSHALL'
```

en cambio, si ponemos:

```
127.0.0.1:6379> c283d93ac9528f986023793b411e4ba2
OK
```

o podemos deshabilitar este comando por completo, haciendo un rename a un string vacío.

```
127.0.0.1:6379> rename-command FLUSHALL ""
```

---

redis-benchmark

¿Cuánto tarda REDIS en hacer 100,000 **X** ?

Donde **X** es [PING_INLINE, PING_BULK, SET, GET, INCR, LPUSH, LPOP, SADD, SPOP, LPUSH (needed to benchmark LRANGE), LRANGE_100 (first 100 elements), LRANGE_300, LRANGE_500, LRANGE_600, MSET]

Para correr este benchmark, tenemos que tener instalado las **redis-tools**.

```
$ sudo apt-get install redis-tools
```

```
$ redis-benchmark -n 100000
...
====== SET ======
  100000 requests completed in 1.37 seconds
  50 parallel clients
  3 bytes payload
  keep alive: 1
```

```
98.63% <= 1 milliseconds
99.83% <= 2 milliseconds
99.86% <= 3 milliseconds
99.92% <= 4 milliseconds
99.93% <= 5 milliseconds
99.94% <= 6 milliseconds
99.95% <= 9 milliseconds
99.97% <= 10 milliseconds
100.00% <= 10 milliseconds
72886.30 requests per second

====== GET ======
  100000 requests completed in 1.35 seconds
  50 parallel clients
  3 bytes payload
  keep alive: 1

99.66% <= 1 milliseconds
99.90% <= 6 milliseconds
99.93% <= 7 milliseconds
100.00% <= 7 milliseconds
74074.07 requests per second
...
```

Master-Slave Replication

[master slave replication discussion redis]

```
$ cp redis.conf redis-01.conf
```

En el `redis-01.conf` (el esclavo o la réplica nro 1), cambiamos el port y avisamos quien es nuestro maestro.

```
file: redis-01.conf
port 6380
replicaof 127.0.0.1 6379
o
slaveof 127.0.0.1 6379
```

`SLAVEOF` (version 4, cuando se escribió el libro)
`REPLICAOF` (version 5, en esta nueva versión, el parámetro cambio de SLAVE a REPLICA)

```
replicaof 127.0.0.1 6379
```

```
y levantamos el esclavo, de la siguiente manera:
```

```
$ redis-server redis-01.conf
```

en el master insertamos

```
127.0.0.1:6379> SADD meetings "StarTrek Pastry Chefs" "LARPers Intl."
```

y en el slave, deberíamos poder ver lo que recién se insertó.

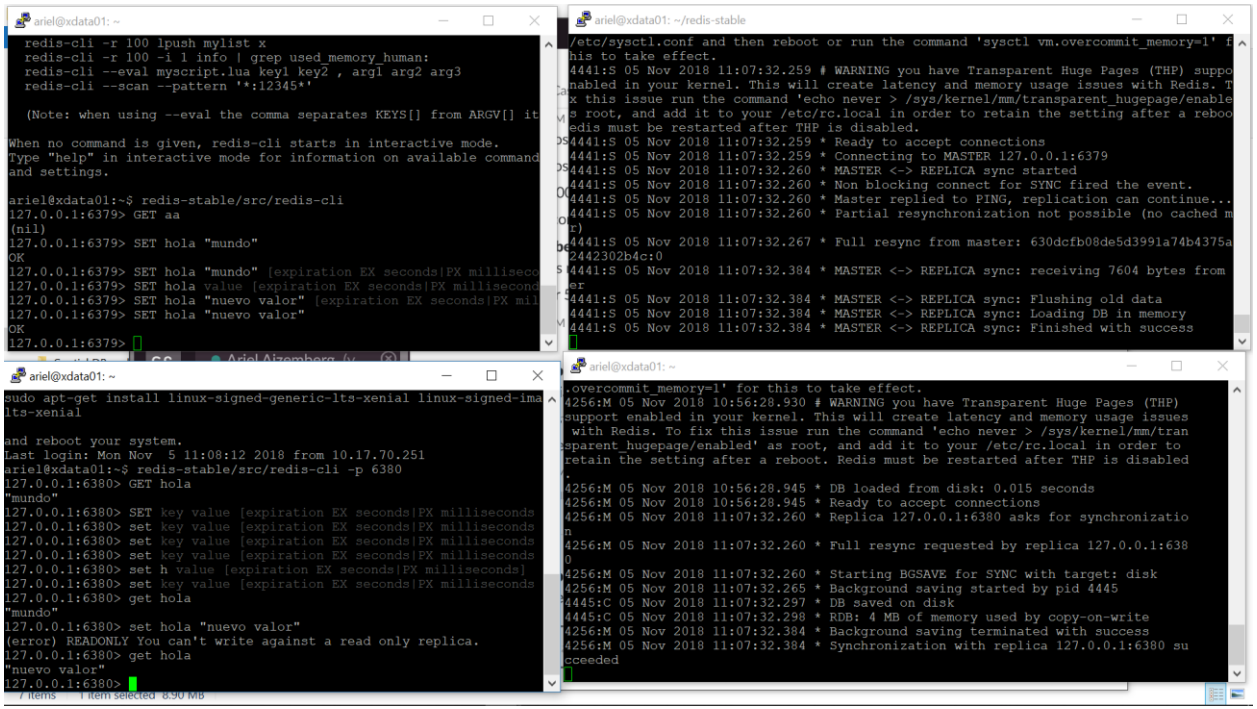
```
127.0.0.1:6380> SMEMBERS meetings
1) "StarTrek Pastry Chefs"
2) "LARPers Intl."
```

Notar que el puerto 6379 es el *master* y el puerto 6380 pertenece al *slave* o la replica.

***In production**, you'll generally want to implement replication for availability or backup purposes and thus have Redis slaves **on different machines**.*

Para armar esta prueba, hay que levantar 2 servers y 2 clientes.

`$ redis-stable/src/redis-server`	Servidor redis en el puerto default, el 6379. (master)
`$ src/redis-server ./redis-01.conf`	Réplica (slave), en el puerto 6380.
`$ redis-stable/src/redis-cli`	Cliente en el puerto default (cliente que trabaja sobre el master, en el puerto 6379)
`$ redis-stable/src/redis-cli -p 6380`	Cliente en el puerto 6380, este cliente no admite escritura, solo lectura y recibe todos los cambios que se realizan en el master.



Cargando datos desde ruby.

`file: redis/data_dump.rb`

```
$ ruby data_dump.rb 100000
100000 items in 5.85 seconds
```

No obstante, se podrían mejorar las inserciones si las operaciones las hacemos con un pipeline.

```
file: redis/data_dump_pipelined.rb

$ ruby data_dump_pipelined.rb 100000
100000 items in 1.06 seconds
```

Redis Cluster

```
file: redis/data_dump_cluster.rb
require 'redis'
require 'redis/distributed'
...
$redis = Redis::Distributed.new([
  "redis://localhost:6379/",
  "redis://localhost:6380/"
])
...

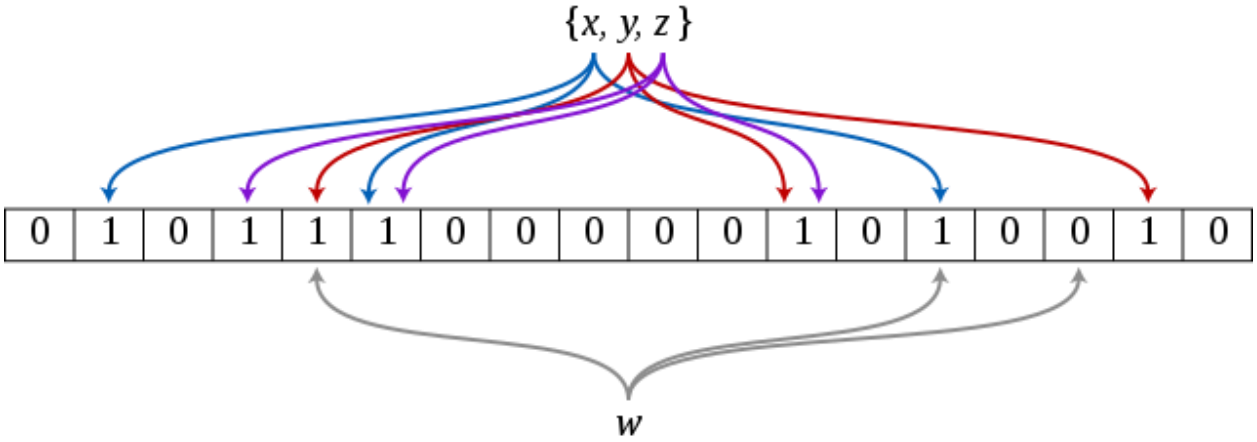
$ ruby data_dump_cluster.rb 100000
100000 items in 6.614907 seconds
```

Antes tardamos con un servidor 5.85 segundos y ahora con un master y un slave 6.61 seg.

Para verificar que un registro fue insertado en las dos instancias

```
$ redis-cli -p 6379 --raw GET key537
$ redis-cli -p 6380 --raw GET key537
```

Bloom Filters



A Bloom filter is a space-efficient probabilistic data structure, conceived by Burton Howard Bloom in 1970, that is used to test whether an element is a member of a set.
https://en.wikipedia.org/wiki/Bloom_filter

Los Bloom Filter son un tipo de datos que no viene por default en Redis (a partir de la v4 sí).

```
$ gem install bloomfilter-rb

file: redis/bloom_filter.rb

$ curl -o moby-dick.txt https://www.gutenberg.org/files/2701/old/moby10b.txt
$ ruby bloom_filter.rb moby-dick.txt > output.txt

...
 if bloomfilter.include?(word)
...
```

Hasta acá, estamos usando los **filtros bloom** por afuera, es decir en la librería **rb** o a través de los comandos GETBIT y SETBIT. A partir de la versión 4.0 de redis, este tipo de datos se puede agregar compilando un módulo. (lista completa de módulos: https://redis.io/modules, por ej. https://github.com/RedisBloom/RedisBloom)

Watch Cristian Castiblanco from Scopely describe its use of RedisBloom → Deduplicating Data Streams with Bloom Filters

https://redislabs.com/redis-best-practices/bloom-filter-pattern/

Building and Loading ReBloom
In order to use this module, build it using **make** and load it into Redis.

Loading
Invoking redis with the module loaded

```
$ redis-server --loadmodule /path/to/rebloom.so
```

Quick Start Guide for ReBloom
Building and running

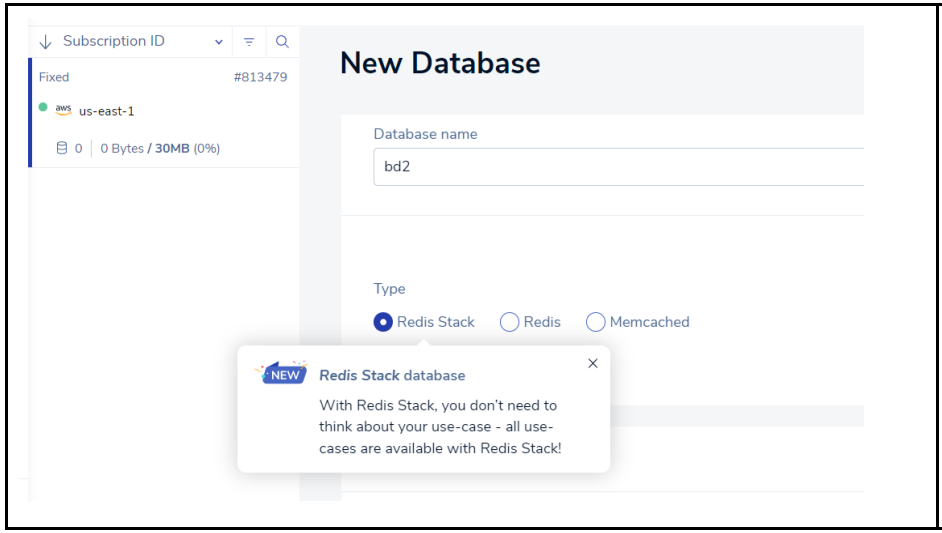
```
$ git clone https://github.com/RedisLabsModules/rebloom.git
$ cd rebloom
$ make

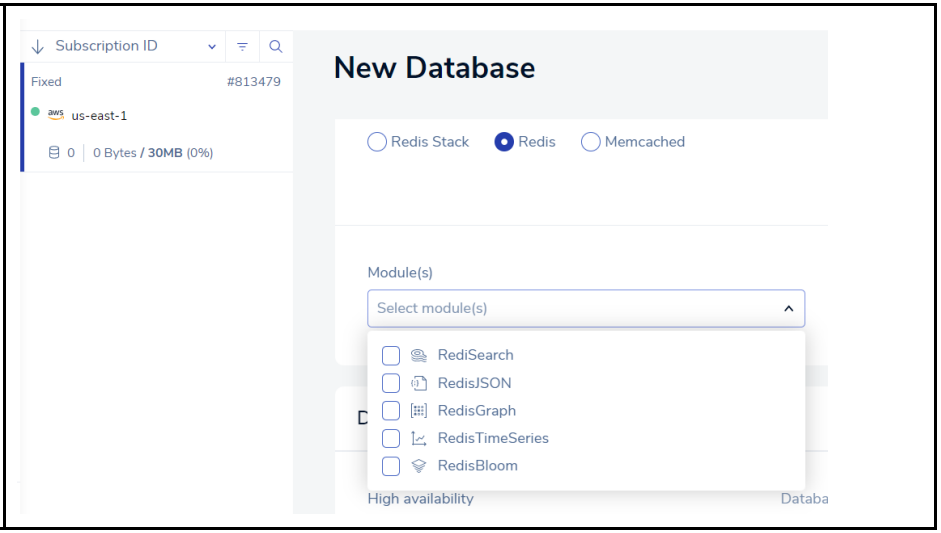
# Assuming you have a redis build from the unstable branch:
/path/to/redis-server --loadmodule ./rebloom.so
```

https://redis.io/docs/stack/bloom/

Redis en la nube, accediendo desde un interactive python notebook en Colab.

https://app.redislabs.com/#/bdbs (mis bases de datos)
https://colab.research.google.com → find: **redis.ipynb**





Redis's Strengths

The **obvious strength** of Redis **is speed**, like so many key-value stores of its ilk. But more than most key-value stores, Redis provides	La **fortaleza obvia** de Redis es la **velocidad**, como tantas bases de datos del estilo clave-valor. Pero más que la mayoría de las bases de datos clave-valor, Redis ofrece la capacidad de almacenar valores

the ability to store complex values such as lists, hashes, and sets, and retrieve them based on operations specific to those data types. Beyond even a data structure store, however, Redis's durability options allow you to trade speed for data safety up to a fairly fine point. Built-in master-slave replication is another nice way of ensuring better durability without requiring the slowness of syncing an append-only file to disk on every operation. Additionally, replication is great for very high-read systems.	complejos como listas, hashes y conjuntos, y recuperarlos en función de operaciones específicas de esos tipos de datos. Sin embargo, más allá de incluso un *key-value store*, las **opciones de durabilidad** de Redis le permiten cambiar la **velocidad** por la **seguridad** de los datos hasta un punto bastante fino. La replicación maestro-esclavo incorporada es otra buena manera de garantizar una mejor durabilidad sin requerir la lentitud de sincronizar un archivo de solo agregar al disco en cada operación. Además, la replicación es excelente para sistemas con alta carga en las lecturas.

Redis's Weaknesses

Redis is fast largely because it resides in memory. Some may consider this cheating because, of course, a database that never hits the disk will be fast. A main memory database has an inherent durability problem; namely, if you shut down the database before a snapshot occurs, you can lose data. Redis also does not support datasets larger than your available RAM (Redis is removing virtual memory support), so its size has a practical limitation. Although there is a Redis Cluster currently in development to grow beyond a single machine's RAM requirements.	Redis es rápido en gran medida porque reside en la memoria. Algunos pueden considerar esta trampa porque, por supuesto, una base de datos que nunca llega al disco será rápida. Una base de datos en memoria principal tiene un problema de durabilidad inherente; a saber, si cierra la base de datos antes de que se decida persistir los datos, se podrían perder algunos datos. Redis tampoco admite conjuntos de datos más grandes que su RAM disponible (Redis está eliminando el soporte de memoria virtual), por lo que su tamaño tiene una limitación práctica. Aunque hay un Redis Cluster actualmente en desarrollo para crecer más allá de los requisitos de RAM de una sola máquina.

Parting Thoughts

Redis is chock-full of commands—more than 120 of them. Most commands are straightforward enough to understand by their names	Redis está repleto de comandos, más de 120 de ellos. La mayoría de los comandos son lo suficientemente simples como para entenderlos solo por sus nombres, una vez que se acostumbra a la idea de que

alone, once you get used to the idea that seemingly random letters will be removed (for example, INCRBY) or that mathematical precision can sometimes be more confusing than helpful (for example, ZCOUNT, or sorted set count, vs. SCARD, or set cardinality). Redis is already becoming an integral part of many systems, …... Regardless of the database you choose as your **SOR** (*system of record*), you should certainly add Redis to the mix (*polyglot ecosystem*).	se eliminarán letras aparentemente aleatorias (por ejemplo, INCRBY) o que la precisión matemática a veces puede ser más confusa que útil (por ejemplo, ZCOUNT o recuento de conjuntos ordenados, frente a SCARD o cardinalidad de conjuntos). Redis ya se está convirtiendo en una parte integral de muchos sistemas ... ... Independientemente de la base de datos que elija como su **SOR** (sistema de registro), sin duda debe agregar Redis a la mezcla (*ecosistema políglota*).

CAP Theorem

Redis y **Neo4J** son consistentes y disponibles (**CA**); no distribuyen datos (*data sharding*), por lo que la partición no es un problema (aunque podría decirse que CAP no tiene mucho sentido en sistemas no distribuidos).

REDIS BEST PRACTICES
https://redislabs.com/community/redis-best-practices/